VISION & IMPLEMENTATION PLAN

תכנית פיתוח מסכמת

מערכת ניסויים מודולרית לדינמיקה, הערכה, חיזוי ובקרה — עם ממשק מחקרי פשוט ושחזור מלא.

עקרון על

מהנדס מתרכז במתמטיקה ובתכנון הניסוי.

התשתית אחראית על חיבור אלגוריתמים לתפקידים, ולידציה, הרצה, מדידה ושחזור.

הגדרת המטרה

לא לפתח רובוט שעוקב באופן מושלם — אלא לבנות מעבדה שמאפשרת לבדוק **אילו מודלים, אלגוריתמים ופרמטרים משפרים מעקב, באילו תנאים ומדוע.**

החלטות ארכיטקטוניות שסיכמנו

01 · YAML

אלגוריתמים עצמאיים מתפקיד + הגדרות ניסוי שמורות

02 · JSON

מטא־נתונים, מדדים, סטטוסים והיסטוריית הרצות

03 · Dashboard

ממשק לניהול ניסויים, לא מנוע נוסף

04 · Reproducibility

Experiment → Revision → Run → Session

גבולות התכנית

עדיין לפיתוח

Simple Experiment Authoring, Mathematical APIs קצרים, רישום נוח, Dashboard, Session Comparison ותצוגת היסטוריה עשירה.

כבר קיים בבסיס המעודכן

Component Registries, ברירות מחדל בחבילות, ExperimentCatalog, הרכבות, בדיקות תאימות, Snapshot והרצת ניסויים Headless.

סטטוס: מסמך יעד לבחינת שתי משימות Codex הבאות; הוא אינו טוען שהן כבר הושלמו.

תוכן התכנית

02 ערכי ליבה · 03 ארכיטקטורה · 04–05 Estimation גנרי · 06 הרכבת ניסוי · 07 תוצאות והיסטוריה · 08 09 · Dashboard מדדים · 10 תכנית ביצוע · 11 מבחני קבלה

DESIGN PRINCIPLES

התאמה לשבעת ערכי הליבה

כל החלטה במימוש תיבחן מול הערכים שהוגדרו לפרויקט, ולא רק מול אסתטיקת הקוד.

ערך	החלטה תכנונית	מבחן קבלה
1. Modularity	Simulation, Estimation, Prediction, Control ו־Experiments בעלי אחריות נפרדת.	מודל חדש לא מחייב עריכה בחבילות לא קשורות.
2. Flexibility & Extensibility	Interfaces + Registries + Schemas + Capabilities; בחירה חיצונית של רכיבים.	מודל תואם חדש ניתן לבחור בלי לשנות Runner / UI.
3. Generality & Reusability	מסנן גנרי נפרד מ־Role, מודל תהליך/מדידה ו־Deployment adapter.	KF/EKF אינו קשור ל־Robot/Target; התאימות נבדקת בנפרד.
4. Configuration-Driven	אלגוריתמים ב־Owner YAML; שיוך לתפקיד, מודלים ו־Overrides רק בהרכבה.	שינוי בחירת IMM/CV לא מחייב שינוי קוד או Default.
5. Clean & Professional	YAML קריא; Authoring קצר; שימוש חוזר במנגנונים קיימים; בלי Catalog כפול.	מהנדס חדש מגדיר ניסוי קצר בלי ללמוד מתכוני Resource פנימיים.
6. Mathematical Rigor	מצב, יחידות, סדר וקטורים, מסגרות ייחוס, זמני חיזוי ואילוצים מפורשים.	שגיאת ממדים / יחידות / תאימות נכשלת לפני הרצה.
7. Testability & Reliability	בדיקות יחידה, שקילות תצורות, Regression, Provenance ו־Immutable Runs.	שינויי UX אינם משנים משוואות, Limits או תוצאות היסטוריות.

אינווריאנטים שאסור לשבור

- ✓ אין רישום מודלים באמצעות Imports שרירותיים מ־YAML
- ✓ אין מנוע ניסויים נוסף בדאשבורד
- ✓ אין דריסה של Revision או Run היסטורי
- ✓ אין שינוי ברירות מחדל עקב ניסוי
- ✓ אין ערבוב בין איכות מעקב לבין תקינות הרצה
- ✓ אין כיוונון ספי קבלה כדי להשיג PASS

דגש תכנוני: פשטות המהנדס דורשת הפרדה בין אלגוריתם, מודל ותפקיד; זו תשתית מדויקת — לא של ויתור על ולידציה, על גבולות מתמטיים או על יכולת שחזור.

ARCHITECTURE

מבנה המערכת וגבולות אחריות

שומרים על המימוש הקיים לאחר קונסולידציה; מוסיפים שכבות שימוש דקות בלבד.

Package-owned model defaults

estimation.yaml (KF/EKF, role-neutral) · prediction.yaml · control.yaml · dynamics.yaml

↓

Simple Experiment Authoring API

בחירת Robot/Target roles + model bindings; Overrides, Preview, Validate, Save

↓

Existing ExperimentCatalog + Composition + M07 Preflight

מקור סמכות אחד · אימות algorithm + state/measurement + role · Revision

↓

Existing ROS 2 / Gazebo / Experiment Runner

בלתי משתנה · הרצה · טלמטריה · חישוב מדדים Snapshot

↓

Results API + Per-run JSON + Session / History Views

תוצאות, גרפים, השוואות, שחזור ומידע מחקרי

מה כן?

לקרוא Component Metadata, לערוך הגדרת ניסוי, לקרוא ל־API הקיים, להציג סטטוס והרצות, לנהל Session ולהציג מדדים שנמדדו ונשמרו.

מה אסור לדאשבורד לעשות?

לבצע חישובי פיזיקה, חיזוי או בקרה; לפרש YAML באופן עצמאי; לחשב מדדי איכות חדשים; לנהל Registry חלופי; או לכתוב ישירות לקובצי ברירת המחדל.

הפרדה קריטית: המהנדס יכול לערוך YAML של מודל כדי לשנות Default. ניסוי מסוים משנה ערכים דרך Overrides שלו בלבד. שיוך KF/EKF לתפקיד Robot/Target נעשה ב־Experiments, לא ב־YAML של האלגוריתם.

MODEL CONFIGURATION

Estimation: אלגוריתם אינו תפקיד בניסוי

היעד המעודכן: KF/EKF מוגדרים פעם אחת; הניסוי קובע אם להפעיל אותם על Robot או Target.

experiments.yaml · בחירה ושיוך לתפקיד

```yaml
experiment:
  name: "Tracking Experiment 01"
  components:
    robot_estimator: ekf
    target_estimator: kf
    predictor: imm
    controller: jacobian
    target_dynamics: fight_flight

# Existing base case supplies compatible
# process/measurement model bindings.
# User overrides are experiment-local.
```

זהו Schema יעד, לא מצב קיים. כיום המזהים התקפים הם robot_ekf ו־kalman. אין להחליפם בקיצור שמות בלבד או לשבור גרסאות היסטוריות.

estimation.yaml · ספריית אלגוריתמים (יעד)

```yaml
estimators:
  kf:
    parameters: {}
    # generic KF-specific options

  ekf:
    parameters: {}
    # generic EKF-specific options

  # Future:
  # ukf:
  #   parameters: {}

# No robot_estimators / target_estimators
# as algorithm-family selectors.
# No selected estimator here.
```

הפרדה תלת־שכבתית שחשוב לאכוף

מה מוגדר?	איפה והמשמעות
Algorithm	estimation.yaml: KF, EKF, UKF עתידי; רק פרמטרים והיכולות של האלגוריתם הגנרי.
Model / Adapter	מצב, F/f, תצפית H/h, חיישנים, Q/R והאתחול לפי מודל/תפקיד; ללא שכפול לוגיקת המסנן.
Role / Selection	experiments.yaml: שיוך אלגוריתם לתפקיד robot_estimator או target_estimator, עם Overrides מקומיים.

העיקרון: אותה משפחת EKF יכולה לשרת כמה תפקידים **רק אם** מודלי התהליך והמדידה והממשקים תואמים. אין לפרש שינוי שמות ב־YAML כהוכחה שתמיכה רב־תפקידית כבר קיימת.

ESTIMATOR CONTRACTS & MIGRATION

איך מיישמים Estimation גנרי בבטחה?

האלגוריתם חוזר לשימוש; תיאור המערכת והחיישנים נשארים ספציפיים לתפקיד.

Generic KF / EKF algorithm
predict + update + covariance propagation; role-neutral numerical core

↓

Process + measurement models
state layout · f/F · h/H · Q/R assumptions · initialization · time/frame contracts

↓

Role adapter + capability validation
robot sensors / target detections · compatible dimensions · deployable output

↓

Experiment selection
robot_estimator: ekf · target_estimator: kf · optional role-specific overrides

בחירה פשוטה; פירוט רק כשצריך

מקרה מתקדם · Binding מפורש (דוגמה)

```
estimation_bindings:
  robot:
    process_model: robot_motion
    measurement_model: wheel_imu
  target:
    process_model: target_cv_3d
    measurement_model: target_position
```

מקרה רגיל · Base Case מספק Bindings

```
components:
  robot_estimator: ekf
  target_estimator: kf
  predictor: imm
  controller: jacobian
```

Migration & backward compatibility

1 · Audit	להפריד את ליבת המסנן הקיימת מלוגיקת wheel/IMU/joints ומ־PointStamped; לא להניח שהיא כבר גנרית.
2 · Implement	מסנן גנרי ו־Role Adapters דקים, ללא שינוי במשוואות או בפרמטרים הנומריים.
3 · Validate	בדיקות תאימות מצב, חיישנים, יחידות, ממדים, Q/R ו־Deployment לפני הרצה.
4 · Migrate	מיפוי מזהים robot_ekf / kalman לייצוג החדש, עם קריאה נכונה של ניסויים ורשומות היסטוריות.
5 · Prove	שקילות נומרית/תצורתית + מבחן שימוש אמיתי באותו אלגוריתם בכמה תפקידים, רק כשהם תואמים.

גבול ביצוע: Prompt A/B שכבר נשלחו אינם מחייבים את המיגרציה הזו. זהו **Milestone ייעודי** לאחר קבלת תוצאותיהם. לפני השלמתו ממשיכים להשתמש במזהים וב־Adapters התקפים כיום.

EXPERIMENT AUTHORING

קובץ ניסוי פשוט, שנשמר בקטלוג קנוני

שכבת Authoring קצרה למהנדס; מאחוריה ה־ExperimentCatalog הקיים שומר גרסאות והרכבות.

דוגמה למבנה רצוי לממשק Authoring — לרבות מזהי Estimator גנריים שטרם מומשו

```yaml
schema_version: 1
experiment:
  id: EXP-TRACK-001
  name: "IMM vs CV - baseline"
  base_case: nominal_tracking

  components:
    robot_estimator: ekf   # future role-neutral ID
    target_estimator: kf   # future role-neutral ID
    predictor: imm
    adaptation: none
    fusion: none
    controller: weighted_least_squares
    target_dynamics: fight_flight

  overrides:
    target_dynamics:
      damping: 0.23

  execution:
    duration: 30.0
    seeds: [42, 43, 44]
```

פשטות חיצונית ≠ החלפת מנוע: המסמך הקצר מתורגם ל־Canonical Definition + Overrides הקיימים.

כללי שמירה והרכבה

1 · Discover	הצגת המודלים הרשומים לפי תפקיד ולפי יכולת פריסה אמיתית.
2 · Compose	Base Case + בחירות מפורשות + Overrides. אין שכפול ברירות מחדל.
3 · Validate	בדיקות Schema, ממדים, קלטים, שדות, Actuation ו־M07 Preflight.
4 · Save Revision	שמירה דרך API בקטלוג הניסויים, בלי עריכה ישירה של מאגר פנימי מורכב.
5 · Freeze & Run	יצירת Resolved Snapshot בלתי משתנה ומזהה Run חדש לכל הרצה.

השם המומלץ: להשאיר experiments.yaml כקטלוג קנוני (קיים כיום, כולל היסטוריה ו־Legacy). ה־Base Case קובע גם Bindings תקינים של תהליך/מדידה. מהנדס חדש עובד מול מסמך Authoring קצר או דרך הדאשבורד; ה־UI אינו משכתב את קובץ הקטלוג ישירות.

RESULTS & REPRODUCIBILITY

YAML להרכבה · JSON לתוצאות · Session להשוואה

להפריד בין הגדרת ניסוי, ביצוע פיזי והצגת היסטוריה, בלי ליצור מאגר תוצאות מתחרה.

SESSION	RUN	REVISION	EXPERIMENT
SESSION-004	**RUN-017**	**r3**	**EXP-TRACK-001**
קבוצת הרצות להשוואה	ביצוע מסוים, Seed ו־Snapshot	איזו גרסה שמורה הופעלה?	מה נבדק? רכיבים ותנאים

אחסון תוצאות — ללא JSON ענקי אחד

```
results/
  studies//
    experiment.yaml
    resolved_experiment.json
    summary.json
    comparison.csv
    runs//
      inputs/
      status.json
      metrics.json
      time_series.json
      telemetry/

  history/index.json  # optional
  sessions/.json   # optional
```

JSON · Run Summary (תבנית שדות מוצעת)

```
{
  "run_id": "RUN-017",
  "experiment_id": "EXP-TRACK-001",
  "revision": 3,
  "session_id": "SESSION-004",
  "started_at": "YYYY-MM-DDThh:mm:ss+TZ",
  "execution_status": "completed",
  "acceptance_status": "conditional",
  "metrics": {
    "distance_mean_error_m": null,
    "pointing_mean_error_deg": null
  },
  "snapshot_path": "inputs/system.yaml",
  "results_path": "runs/RUN-017"
}
```

היסטוריה: כבר קיים Backend לשמירת Run IDs ותוצאות. history/index.json הוא אינדקס נגזר ואופציונלי לצורכי UI, לא Source of Truth שני.

תיעוד מחקרי (Backend מתוכנן, לא רק Dashboard)

לפני ההרצה: שאלת מחקר, Hypothesis, משתנים בלתי תלויים ומשתנים מבוקרים. לאחריה: תצפיות ומסקנות עם הפניות ל־Revision/Run IDs. התיעוד יתועד כרשומות גרסאות/הערות נפרדות, בלי לשכתב Metrics או Snapshots היסטוריים.

כללי מדע ונתונים

- ✓ Revision היסטורי לא נדרס
- ✓ כל Run מקבל מזהה ייחודי
- ✓ Missing Metric מוצג כחסר ולא כאפס
- ✓ Seeds, מקור הקוד ו־Snapshots נשמרים
- ✓ אפשר לשחזר את ההרכב גם ללא Dashboard
- ✓ Session רק מקבץ Runs; אינו משנה תוצאות

DASHBOARD WORKFLOW

התנהגות הדאשבורד — משלב ההכנה ועד ההיסטוריה

יישום הדרגתי ב־Plotly/Dash כשכבת UI קלה מעל השירותים הקיימים.

01

Simulation Setup

פתיחת Gazebo, בחירת World/Robot, אימות התאמה לתצורת הניסוי. החלפת סביבת פריסה מחייבת Restart / Reconfiguration מבוקר.

02

Build / Select Experiment

יצירת ניסוי קצר או בחירת ניסוי שמור. בחירת Sensors; Reference, Dynamics, Controller, Predictor, Estimator; עריכת Overrides.

03

Preview → Validate → Save

הצגת ערכים אפקטיביים ושינויים מול Defaults; בדיקות תאימות; יצירת Revision; פעולה מפורשת להפעלת Actuation.

04

Run & Monitor

הפעלה, עצירה, סטטוס Telemetry, Readiness ושגיאות. אין הסתרת כשלי Clock/Readiness או כיסוי חלקי.

05

Results → Current Session Comparison

מיד בסיום: גרפים ומדדים של Run. הוספה לטבלת Session הנוכחי; ממשיכים לניסוי הבא באותו Session.

06

Historical Experiments & Replay

סינון לפי זמן, מודל, Experiment, Revision ו־Session; פתיחת Snapshot, השוואה והרצה חדשה מאותו הרכב.

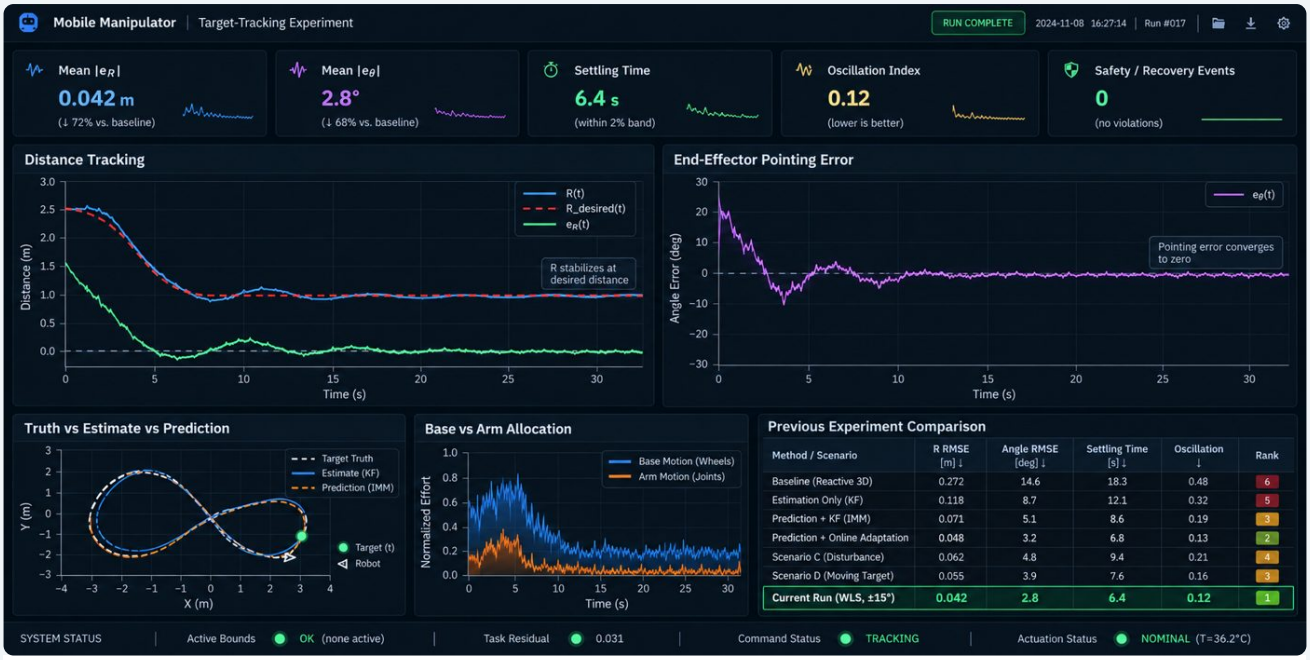
Experiment Builder	Live & Results	History & Comparison

דרישת שימושיות: אין צורך לערוך base/group/resources/selection ידנית. כל המורכבות הזו נשארת ב־Backend. ממשק Headless מספק אותן פעולות ללא GUI.

ANALYSIS & VISUAL LANGUAGE

תוצאות וגרפים — אסתטיקה עם משמעות מחקרית

תמונת הדאשבורד שצורפה משמשת השראה חזותית בלבד; כל נתון שיוצג בעתיד יגיע מהרצה אמיתית.



איור ייחוס שסופק על ידי המשתמש · המספרים המוצגים בו אינם מדדים מאומתים של הרצות הפרויקט.

משפחות מדדים להצגה

קטגוריה	הצגה מומלצת	זהירות מדעית
Tracking	מרחק רצוי מול בפועל; שגיאת מרחק; שגיאת כיוון; זמן התייצבות.	להגדיר מדויק Time Window ואופן חישוב Settling.
Estimation	Ground Truth מול Estimate; Error Time Series; Covariance כשזמינה.	אין לחשוף Ground Truth ל־Controller דרך UI/Composition.
Prediction	Trajectory Forecast; טווחי חיזוי; Error vs Horizon.	יישור Timestamp ואופק חובה לפני חישוב שגיאה.
Control & Safety	Base/Arm allocation, פקודות, Limits, Watchdogs, Violations.	PASS בטיחות/תקינות אינו בהכרח Tracking טוב.
Comparison	Runs באותו Session + Historical Studies + Rankings.	לדרג רק הרצות ברות השוואה ובעלות כיסוי תקף.

כלל יסוד: Execution status, Safety acceptance ו־Tracking performance הם שלושה ממדים שונים. כשל Readiness או כיסוי חסר לא הופך אוטומטית למספר אפס או ל־PASS.

IMPLEMENTATION ROADMAP

שלבי פיתוח — מה נבדוק קודם ומה נבנה אחר כך

Prompt A/B משפרים שימושיות; המיגרציה ל־Estimator גנרי היא שלב ממוקד ונפרד, לפני ייצוב בחירת רכיבים ב־UI.

BASE

בסיס ארכיטקטוני קיים — Task 1 + Task 2

20 קובצי קונפיגורציה; Component Registries; Catalog קנוני; Compose/Validation; Snapshots, History ו־Headless API.

דווח כממומש · טעון אימות מול ZIP הבא

A

Prompt A · E1 + M1

YAML קריא מבחינה מתמטית; Simple Experiment Authoring; בחירה, Preview, Validate, Create / Revise ללא מתכוני Composition ידניים.

בהרצה אצל Codex · תוצאות טרם התקבלו

B

Prompt B · M2 + M3

API קצר עבור Acceleration Dynamics, Predictor פשוט ובקר פשוט; רישום מקומי מבוקר; שילוב ב־Registries הקיימים.

יתבצע לאחר A · תוצאות טרם התקבלו

E

חדש — Estimator Algorithm / Role Decoupling

הפרדת KF/EKF מלוגיקת Robot/Target; Bindings למודלי תהליך/מדידה; Capability Validation; Migration למזהים היסטוריים.

לא מכוסה ב־Prompt A/B · לבצע לאחר בדיקת תוצאותיהם

C

Backend לשכבת Dashboard

אימות חסרים ב־Catalog CRUD, Session IDs, History Query, Status/Run API ואינדקס JSON נגזר אם נחוץ. אין Data Store כפול.

D1

Dashboard 1 · Builder + Gazebo Setup

בחירת/יצירת ניסוי; עריכת Defaults ו־Overrides לפי Schemas; Validation; שמירת Revision; פתיחת Simulation.

D2

Dashboard 2 · Run + Current Session

Start/Stop, Live Status, גרפים לאחר סיום, Session Comparison ותוספת Run אוטומטית לטבלה.

D3

Dashboard 3 · History + Analysis

חיפוש, פילוח, השוואת מחקרים, Snapshot/Revision/Diff, שחזור הרכב, רשומות מחקר והסקת מסקנות.

שלבים עתידיים נפרדים: מצב פנימי גנרי למטוטלת כפולה, מסלולים אנליטיים כלליים, History-Based GRU, MPC/LQR מלאים; טיפול ב־Readiness/Clock ובחזרתיות פיזיקלית. לא להכניס אותם לשני הפרומפטים הנוכחיים או למיגרציית Estimation.

ACCEPTANCE & NEXT REVIEW

מבחני קבלה והחלטות להמשך

המסמך ישמש Checkpoint קבוע: כשיגיעו תוצאות Prompt A/B נבדוק בפועל מול הרשימה.

#	מבחן קבלה	ראיית הצלחה נדרשת
01	המודלים ב־YAML מסודרים ומובנים	קריאות Owner Defaults; יעד נוסף: estimators.kf/ekf ללא שיוך Robot/Target (במיגרציה ייעודית).
02	בחירה פשוטה של מודלים לניסוי	מיידית: robot_ekf + kalman; לאחר מיגרציה: robot_estimator: kf / target_estimator: ekf קצר ב־YAML.
03	יצירת Experiment שמור	Catalog → Preview → Validate → Create/Revise דרך Dashboard.
04	הוספת מודל פשוט	מודל Dynamics/Predictor/Controller בדיקתי עם פונקציה מתמטית קצרה, רישום וגילוי דרך אותם APIs.
05	תאימות וממדים	מודל לא תואם נדחה לפני הפעלה, עם הודעה על שדה/חוזה שנכשל.
06	אי־פגיעה בהתנהגות קיימת	Numerical/Configuration Equivalence, Tests, Builds ו־Installed Run כפי שמאפשרת הסביבה.
07	שחזור והיסטוריה	Experiment ID + Revision + Run ID + Snapshot + Seed, בלי שינוי תוצאות קודמות.
08	Session Comparison	Runs נפרדים נרשמים לאותו Session וניתנים להצגה בלי לשכפל את נתוני הריצות.
09	אמינות תוצאות	PASS/FAIL/CONDITIONAL/UNKNOWN, זמינות מדדים וכיסוי מוצגים בנפרד מאיכות Tracking.
10	מסנן בלתי תלוי בתפקיד ותאימות לאחור	רישום KF/EKF גנרי פעם אחת; Bindings תקפים ל־Robot/Target; שקילות מספרית; שמירת מזהי Revisions ו־Runs היסטוריים.

מה להביא לביקורת הבאה אחרי Codex

- ☐ רשימת קבצים ששונו/נוספו/נמחקו
- ☐ ZIP עדכני לאחר Prompt A/B
- ☐ דוגמת פונקציית מודל חדשה והרשמה
- ☐ YAML חדש לדוגמה + CLI יצירת ניסוי
- ☐ Known Issues וסטטוס קבלה אמיתי
- ☐ תוצאות Tests/Equivalence/Builds

כלל ההחלטה: אם שני הפרומפטים משיגים את מבחני 01–07 בלי מנגנונים כפולים או שינוי התנהגות, אפשר להתחיל Dashboard Builder. מבחן 10 הוא Gate נפרד להשלמת Estimation גנרי לפני חשיפת מזהי KF/EKF חוצי־תפקיד; 08–09 יטופלו ב־Backend/Results.

מקורות התכנון: חזון וערכי הליבה שנקבעו בשיחה; ביקורת ה־ZIP המעודכן; דוחות Task 1/2; והחלטת ההפרדה החדשה בין Estimator Algorithm, Model Binding ו־Role. נתוני Baseline שדווחו על ידי Codex אינם מוצגים כאימות ROS/Gazebo עצמאי במסמך זה.